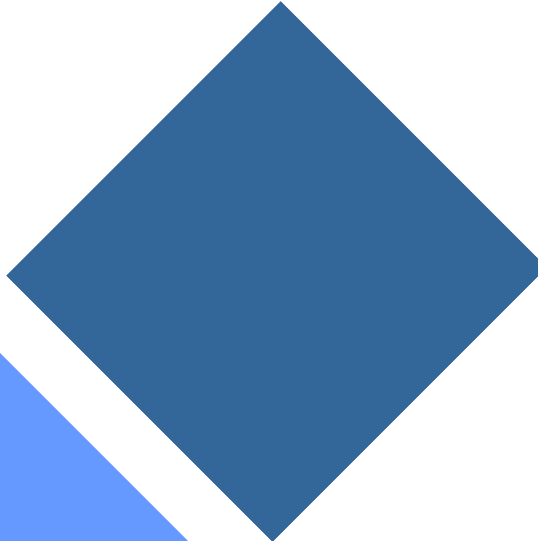


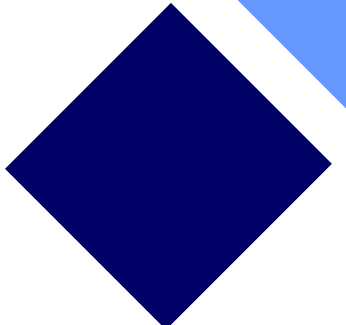
SP3: Ingesta y Lazy Evaluation

Isabel Pelaya Galindo Ibáñez
Esther Ibáñez Mingorance
José Luis López García
Juan Rafael Madolell Usero





Indice



- 1. Schema Definition**
- 2. Lazy Evaluation**
- 3. Transformaciones vs Acciones**
- 4. DAG**
- 5. Problemas**

1. DEFINICIÓN DE SCHEMA

¿Qué es?: Es la definición formal de los nombres de columna y los tipos de datos de un DataFrame.

InferSchema (Inferencia Automática): Spark escanea los datos para adivinar el esquema.

StructType (Definición Manual): Se define el esquema explícitamente mediante código.

1. DEFINICIÓN DE SCHEMA

```
root
|-- student_id: integer (nullable = true)
|-- age: integer (nullable = true)
|-- gender: string (nullable = true)
|-- study_hours_per_day: float (nullable = true)
|-- sleep_hours: float (nullable = true)
|-- phone_usage_hours: float (nullable = true)
|-- social_media_hours: float (nullable = true)
|-- youtube_hours: float (nullable = true)
|-- gaming_hours: float (nullable = true)
|-- breaks_per_day: integer (nullable = true)
|-- coffee_intake_mg: integer (nullable = true)
|-- exercise_minutes: integer (nullable = true)
|-- assignments_completed: integer (nullable = true)
|-- attendance_percentage: float (nullable = true)
|-- stress_level: integer (nullable = true)
|-- focus_score: integer (nullable = true)
|-- final_grade: float (nullable = true)
|-- productivity_score: float (nullable = true)
```

2. LAZY EVALUATION

Spark no ejecuta las operaciones de datos en el momento en que se escriben en el código

¿Cómo funciona?

- Cuando se aplica una transformación, Spark lo anota en un **plan lógico**.
- La ejecución real solo comienza cuando se invoca una **Acción**.

Permite que Spark optimice todo el proceso antes de mover la información, ahorrando memoria y tiempo de CPU.

3. TRANSFORMACIONES VS ACCIONES

Transformaciones: Generan un **nuevo DataFrame** a partir de uno existente. No devuelven datos al usuario.

- Ejemplos: `filter()`, `select()`, `withColumn()`

Acciones: Son el disparador que obliga a Spark a ejecutar el plan lógico acumulado.

- Ejemplos: `show()`, `collect()`, `count()`, `write()`.

3. TRANSFORMACIONES VS ACCIONES

```
[P3 - Científico de Datos]: Realizando transformaciones (Select, Filter, withColumn)...  
- Tiempo Filter: 0.029864s  
- Tiempo withColumn: 0.014742s  
- Tiempo Select: 0.006371s
```

3. TRANSFORMACIONES VS ACCIONES

StructType

 4.0.1

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

SparkRolesABP application UI

Stages for All Jobs

Completed Stages: 5
Skipped Stages: 1

▼ Completed Stages (5)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id ▾	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
5	csv at NativeMethodAccessorImpl.java:0	+details	2026/03/24 15:42:37	68 ms	1/1	1534.2 KiB	86.4 KiB		
4	collect at /opt/spark/work-dir/spark_roles_abp.py:87	+details	2026/03/24 15:42:37	16 ms	1/1	64.0 KiB			
3	showString at NativeMethodAccessorImpl.java:0	+details	2026/03/24 15:42:37	25 ms	1/1	64.0 KiB			
2	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	+details	2026/03/24 15:42:36	28 ms	1/1			59.0 B	
0	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	+details	2026/03/24 15:42:36	0.1 s	1/1	1534.2 KiB			59.0 B

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

▼ Skipped Stages (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id ▾	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	+details	Unknown	Unknown	0/1				

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

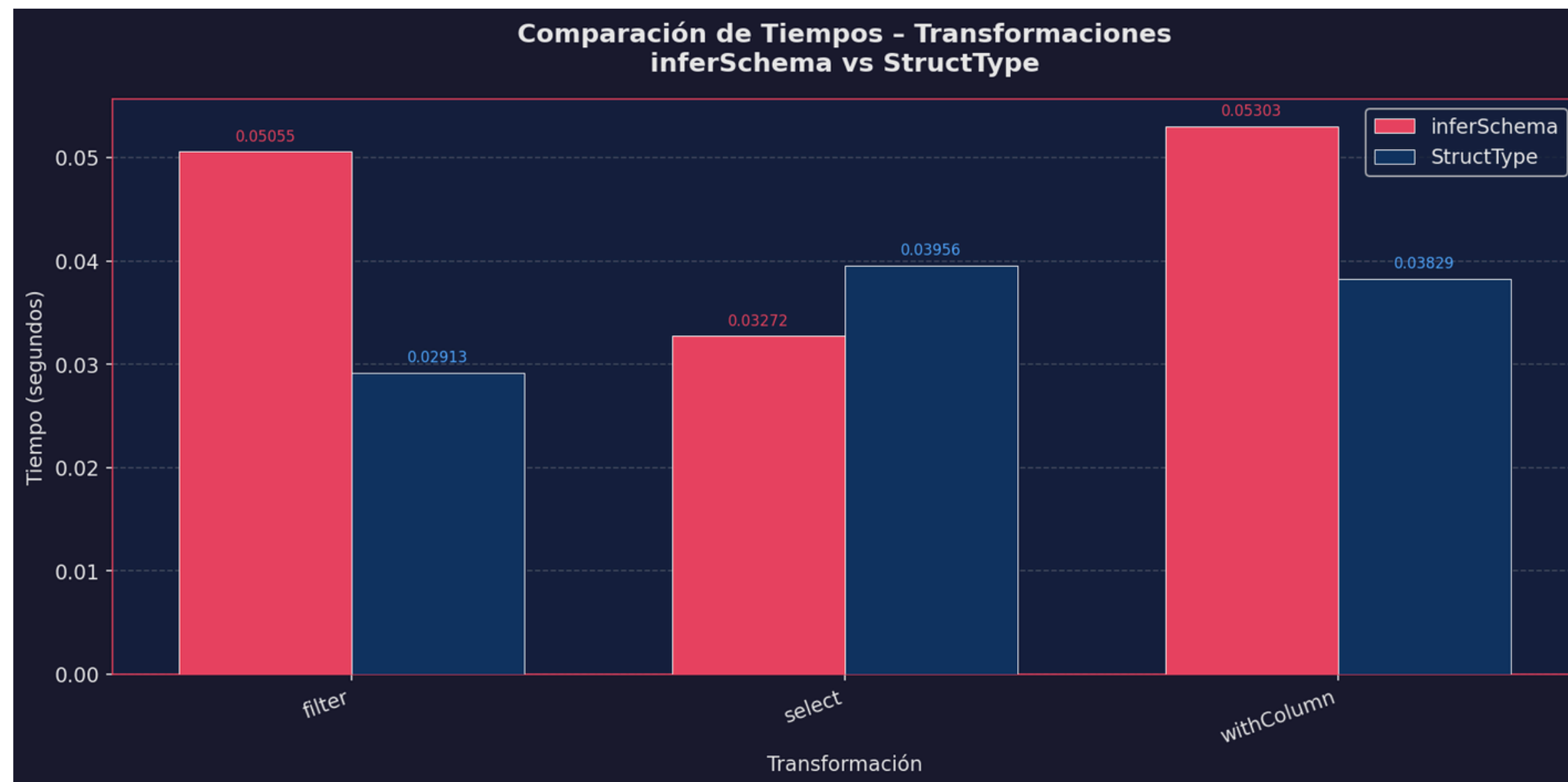
count()

show()

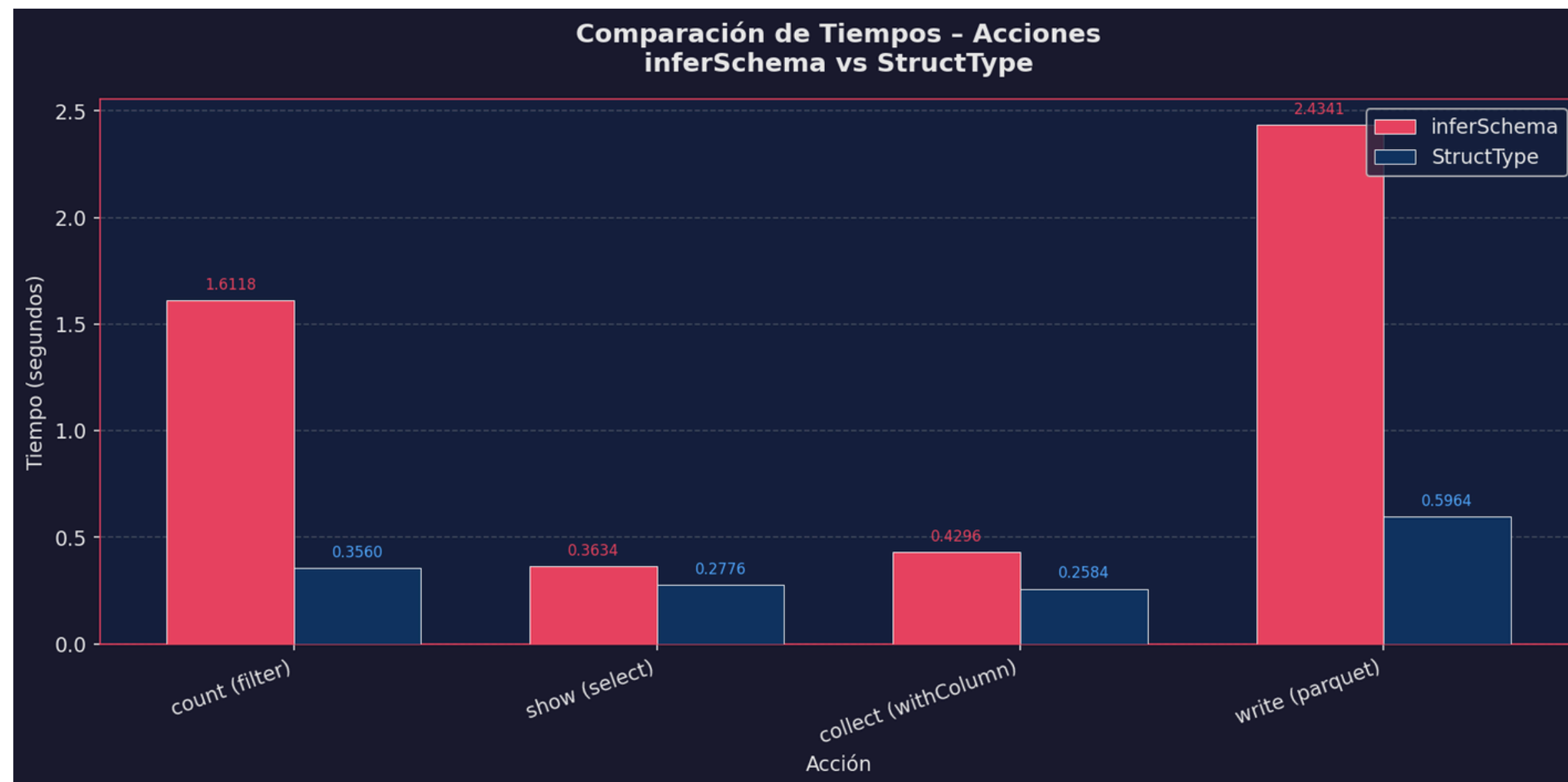
collect()

write()

3. TRANSFORMACIONES VS ACCIONES



3. TRANSFORMACIONES VS ACCIONES



4. DAG

Es un **grafo dirigido acíclico** que representa la secuencia de pasos lógicos que Spark debe seguir para llegar al resultado final. El flujo va en una sola dirección y no hay bucles.

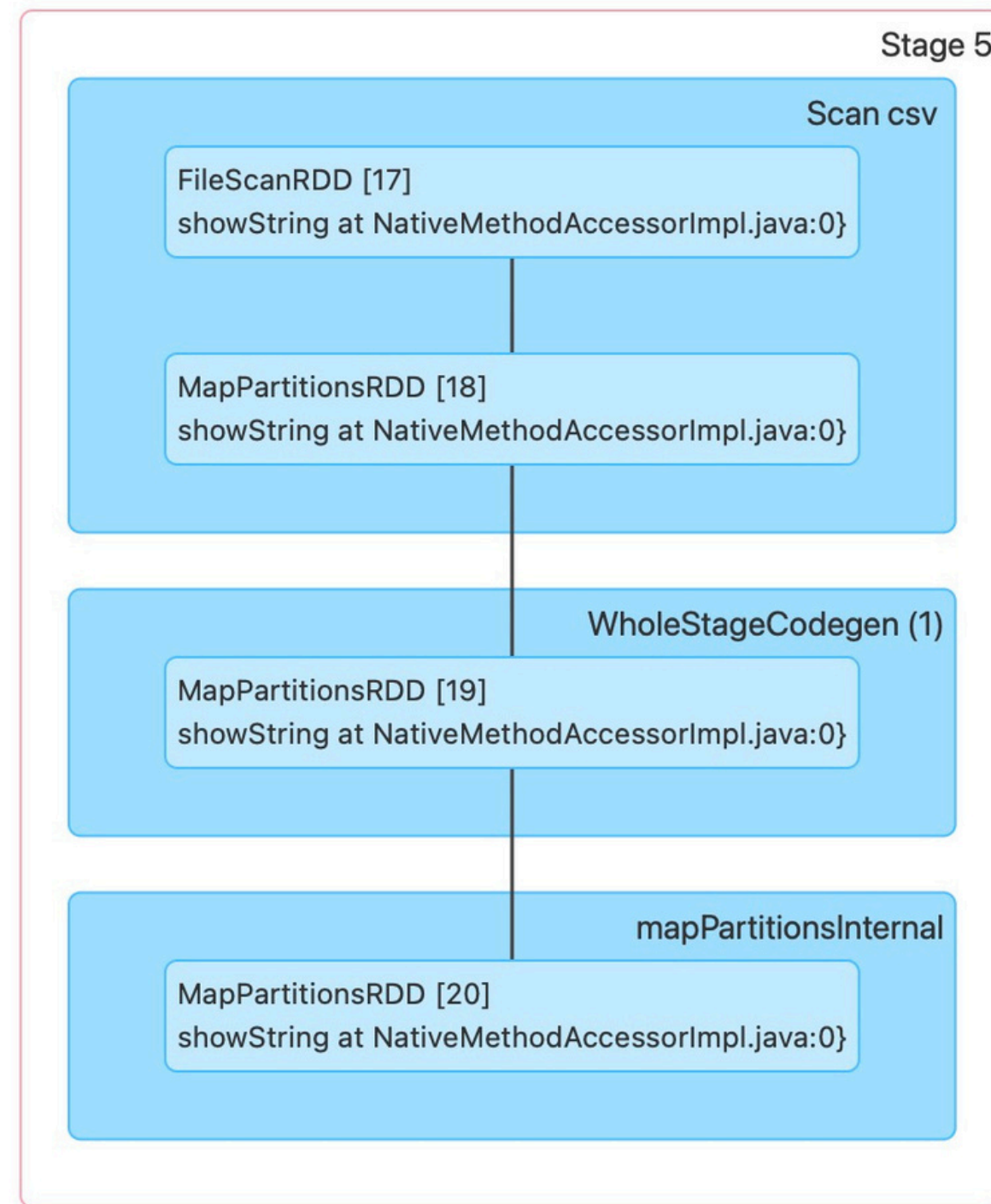
Componentes:

- El DAG permite ver cómo se divide el trabajo en Jobs (el trabajo total) y Stages (etapas de procesamiento).

Si una tarea falla, Spark usa el DAG para re-ejecutar solo la parte perdida, no todo el proceso.

4. DAG

▼ DAG Visualization



5. PROBLEMAS

Acceso a la Spark UI:

El puerto 4040 no estaba accesible desde el navegador del host.

Solución: Se modificó el archivo **docker-compose.yml** para añadir el mapeo del puerto "4040:4040". Se reiniciaron los contenedores para aplicar los cambios, permitiendo el acceso a <http://localhost:4040/>.

5. PROBLEMAS

Persistencia de la sesión para Análisis.

La aplicación de Spark terminaba demasiado rápido, lo que provocaba que la Spark UI se cerrara antes de poder analizar el DAG.

Solución: Se implementó una pausa programada (`time.sleep(120)`) al final del script. Esto mantiene activa la `SparkSession` el tiempo suficiente para realizar las capturas de pantalla y el análisis de rendimiento.

**Muchas
gracias**

